Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления

Кафедра информационных технологий автоматизированных систем

ОТЧЕТ
По практическому занятию №1

«*Middleware* и шаблонизаторы»

Выполнила ст. гр. 320604
К. А. Хаджинова

Проверил А. Л. Гончаревич

Минск 2026

СОДЕРЖАНИЕ

Введение....3
1 Постановка задачи....4
2 Теоретическая часть....5
3 Ход работы....6
4 Ответы на контрольные вопросы....14
Заключение....15
Приложение А (обязательное) Настройка *Express* (*app.js*)....16
Приложение Б (обязательное) Шаблон главной страницы (*index.ejs*)....17
Приложение В (обязательное) Шаблон шапки (*header.ejs*)....18
Приложение Г (обязательное) Логирование запросов (*logger.js*)....19
Приложение Д (обязательное) Журнал запросов (*logStore.js*)....20
Приложение Е (обязательное) Имитация авторизации (*auth.js*)....21
Приложение Ж (обязательное) Маршруты страниц (*pageRoutes.js*)....22
Приложение И (обязательное) Обработка ошибок (*errorHandlers.js*)....23

ВВЕДЕНИЕ

В рамках практического занятия исследуются принципы работы промежуточных обработчиков запросов (*middleware*) в веб-фреймворке *Express.js* и шаблонизатора *EJS*, которые позволяют создавать динамические веб-страницы на стороне сервера.

Express.js – минималистичный веб-фреймворк для платформы *Node.js*, который предоставляет средства маршрутизации, обработки запросов и ответов и подключения промежуточных обработчиков. Любое приложение на *Express* представляет собой цепочку таких обработчиков, через которую последовательно проходит каждый запрос. Благодаря этому сквозные задачи, такие как логирование, проверка прав доступа, разбор тела запроса и обработка ошибок, выносятся из обработчиков маршрутов и переиспользуются во всем приложении.

Шаблонизатор *EJS* позволяет формировать *HTML*-страницы из шаблонов, в которые подставляются данные с сервера. Шаблоны содержат обычную разметку и вставки на *JavaScript*, поэтому страницы создаются динамически и не требуют отдельного клиентского приложения. Общие части страниц, например шапка и подвал, выносятся в частичные шаблоны и подключаются во все страницы.

Практическое занятие выполнено на примере проекта *VisualDSL*, тематика которого соответствует курсовой работе. Приложение хранит данные в массиве в оперативной памяти и использует собственные *middleware* для логирования запросов, имитации авторизации и обработки ошибок.

VisualDSL – платформа для создания визуальных диаграмм по текстовому описанию на языке *DSL*. Каждая диаграмма имеет название, тип (*flowchart*, *sequence* или *state*) и *DSL*-код, а в списке отображается с идентификатором и датой создания. На этой предметной области показано, как *middleware* и шаблоны применяются в типичном веб-приложении.

Запрос клиента проходит по цепочке обработчиков. Сначала он записывается в журнал, затем разбирается его тело, определяется пользователь, и запрос передается маршруту. Если ни один маршрут не ответил, управление получает обработчик ошибки 404, а исключения перехватывает обработчик ошибок с четырьмя параметрами.

В отчете последовательно изложены постановка задачи, теоретические сведения, ход работы с описанием каждой страницы приложения, ответы на контрольные вопросы и заключение.

1 ПОСТАНОВКА ЗАДАЧИ

Целью работы является освоение концепции *middleware* в *Express.js*, разработка собственных *middleware* для логирования, имитации авторизации и обработки ошибок и изучение шаблонизатора *EJS* для генерации динамических *HTML*-страниц на стороне сервера. Тематика проекта соответствует курсовой работе, это платформа для создания визуальных *DSL*-диаграмм (*VisualDSL*), данные которой хранятся в массиве в памяти.

В рамках практического занятия необходимо решить пять задач, каждая из которых получила отражение в ходе работы и в коде проекта:

– создать *Express*-приложение с поддержкой *EJS* и настроить *view engine*;

– реализовать *middleware* логирования метода, *URL* и времени запроса с сохранением записей в журнал, который выводится в виде таблицы;

– реализовать *middleware* имитации авторизации по параметру ?*auth*=1;

– создать динамические страницы: главную, детальную и форму добавления;

– добавить обработку ошибок 404 и 500 с отдельными страницами.

Данные хранятся в массиве в оперативной памяти, база данных не используется. Код размещен в ветке *pz21* репозитория проекта.

После завершения разработки приложение необходимо проверить в браузере и автоматическими тестами, убедившись в корректной работе страниц, формы добавления и каждого *middleware*. Исходный код хранится в системе управления версиями *Git*, а результаты работы фиксируются скриншотами страниц.

Приложение должно предоставлять следующие страницы: главную со списком диаграмм, детальную страницу выбранной диаграммы, форму добавления новой диаграммы, страницу входа, журнал запросов и отдельные страницы ошибок 404 и 500. Форма добавления доступна только авторизованному пользователю, а введенные данные проверяются на стороне сервера. При ошибках проверки форма повторно показывается с введенными значениями и кодом 400.

Журнал запросов должен сохранять время, тип события, метод, *URL*, статус, длительность, имя пользователя и *IP*-адрес каждого запроса и выводиться таблицей с возможностью фильтрации по методу и типу события. Авторизация имитируется параметром ?*auth*=1 в адресе, поэтому запросы без него выполняются от имени гостя.

2 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

Middleware представляет собой функцию, которая получает объект запроса, объект ответа и функцию *next*, обрабатывает запрос и либо передает управление дальше по цепочке, либо завершает ответ. Порядок подключения таких функций определяет поведение всего приложения, поэтому особое внимание в работе уделено последовательности их подключения. Статические файлы и разбор тела запроса подключаются до маршрутов, а обработчики ошибок 404 и 500 – после них.

Различают несколько видов *middleware*. Обработчики уровня приложения подключаются методом *app.use* и срабатывают для всех запросов или для запросов с заданным префиксом пути. Обработчики уровня маршрутизатора работают так же, но привязаны к экземпляру *express.Router*. Встроенные обработчики *Express* включают *express.static* для отдачи статических файлов, *express.json* и *express.urlencoded* для разбора тела запроса. Существуют и сторонние *middleware*, подключаемые из пакетов *npm*.

Особое место занимают обработчики ошибок. Они имеют четыре параметра (*err*, *req*, *res*, *next*) и вызываются, когда в цепочке возникло исключение или в функцию *next* передана ошибка. Такие обработчики подключаются после всех маршрутов. Обработчик ошибки 404 не является обработчиком ошибок в строгом смысле. Он подключается последним среди обычных обработчиков и срабатывает, если ни один маршрут не ответил на запрос.

Шаблонизатор *EJS* позволяет собирать страницы из общих частей и подставлять в них данные из массива диаграмм, при этом вывод через <%= %> экранирует *HTML*.

В шаблонах *EJS* используются три основных вида тегов. Тег <% %> выполняет код без вывода, тег <%= %> выводит значение с экранированием специальных символов *HTML*, а тег <%- %> выводит значение без экранирования и применяется для вставки готовой разметки, например частичных шаблонов через *include*. Экранирование защищает приложение от внедрения скриптов (*XSS*), поэтому данные пользователя выводятся только через <%= %>.

Данные в шаблоны передаются вторым аргументом метода *res.render*. Кроме того, общие значения можно сохранить в *app.locals*, доступных во всех шаблонах, и в *res.locals*, которые существуют в рамках одного запроса. Это удобно для функций-помощников, например форматирования даты, и для информации о текущем пользователе, которую определяет *middleware* авторизации.

3 ХОД РАБОТЫ

Работа выполнена поэтапно. Сначала настроены проект и шаблонизатор, затем созданы шаблоны страниц, реализованы *middleware*, маршруты и обработка ошибок.

В *app.js* задаются параметры *view engine* и *views*, полный код приведен в приложении А.

Общий каркас страницы вынесен в частичные шаблоны *partials/header.ejs* и *partials/footer.ejs*. Шаблон главной страницы приведен в приложении Б, а шапка сайта – в приложении В.

Логирующее *middleware* подписывается на событие *finish* ответа, выводит в консоль метод, *URL*, код состояния и время обработки в миллисекундах и сохраняет запись в журнал *logStore*. *Middleware* авторизации состоит из двух функций. Функция *attachUser* определяет пользователя по параметру ?*auth*=1 и передает его в шаблоны через *res.locals*, а *requireAuth* перенаправляет гостя на /*login*. Код приведен в приложениях Г, Д и Е.

Маршруты страниц описаны в *routes/pageRoutes.js*, а логика обработчиков находится в *controllers/pageController.js*. Код маршрутов приведен в приложении Ж.

Обработчики ошибок подключаются последними. Функция *notFoundHandler* отображает страницу 404.*ejs*, а *serverErrorHandler* имеет четыре параметра, что делает ее обработчиком ошибок *Express*. Она отображает 500.*ejs*. Код обработчиков приведен в приложении И.

Главная страница с таблицей диаграмм из массива показана на рисунке 3.1.

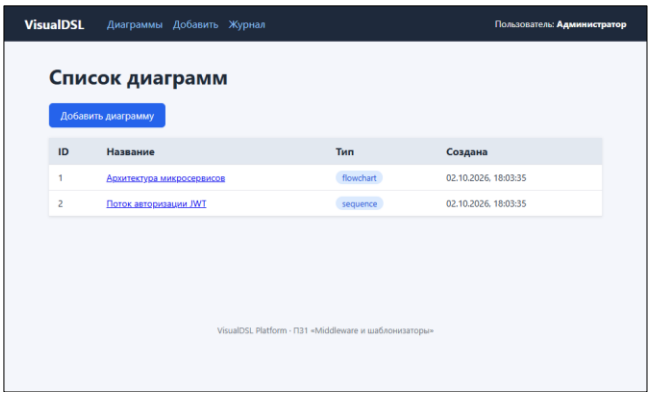


Рисунок 3.1 – Главная страница со списком диаграмм

На рисунке 3.1 видны два начальных элемента массива, а имя пользователя в шапке и параметр ?*auth*=1 в ссылках подтверждают работу

middleware авторизации. Цикл *forEach* в шаблоне *index.ejs* вывел каждый элемент отдельной строкой таблицы. Приложение запущено командой *npm start*, страницы открыты в браузере. В шапке сайта расположены ссылки «Диаграммы», «Добавить» и «Журнал», а справа показано имя пользователя «Администратор». Таблица содержит столбцы «*ID*», «Название», «Тип» и «Создана», а тип диаграммы выделен значком, например *flowchart* и *sequence*. Название каждой диаграммы является ссылкой на ее детальную страницу, а в нижней части страницы выводится подпись с названием практического занятия. Для отображения главной страницы маршрут вызывает метод контроллера, который получает массив диаграмм из хранилища и передает его в шаблон *index.ejs*. Шаблон подключает частичные шаблоны шапки и подвала и выводит строки таблицы циклом *forEach*, а дату создания форматирует функцией *formatDate*. Параметр *auth* в адресе передается во все ссылки через переменную *authQuery*, поэтому авторизация сохраняется при переходах между страницами. При этом в журнале запросов появляется запись об обращении к главной странице со статусом 200, а в консоли сервера выводится строка с методом, адресом, кодом ответа и временем обработки. Начальные данные списка вынесены в папку *seeds/* и содержат два элемента, «Архитектуру микросервисов» с типом *flowchart* и «Поток авторизации *JWT*» с типом *sequence*.

Страница на рисунке 3.2 открывается по адресу */item/1* и выводит поля выбранного элемента.

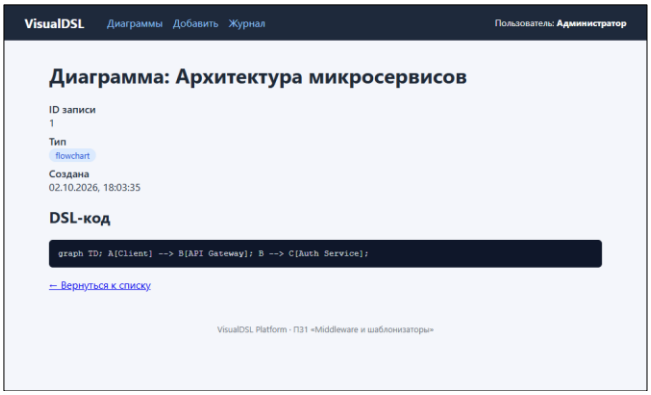


Рисунок 3.2 – Детальная страница диаграммы

Маршрут */item/:id* нашел элемент по идентификатору и передал его в шаблон *item.ejs*. *DSL*-код отображается в экранированном виде, поэтому разметка внутри кода не может выполниться в браузере. На странице выведены идентификатор записи, тип диаграммы *flowchart*, дата и время создания и *DSL*-код в отдельном темном блоке. Ссылка «Вернуться к списку» позволяет перейти на главную страницу. Дата создания форматируется

функцией *formatDate*, которая подключена через *app.locals* и доступна во всех шаблонах.

Форма добавления доступна только после авторизации и показана на рисунке 3.3.

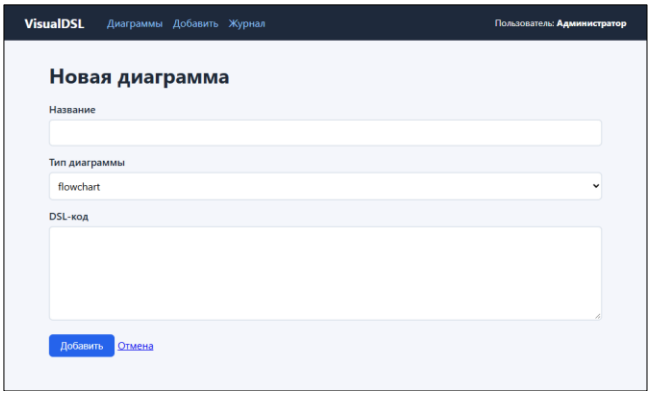


Рисунок 3.3 – Форма добавления диаграммы

Форма содержит поля названия, типа и *DSL*-кода. Адрес отправки */add?auth=1* сохраняет признак авторизации, поэтому после отправки пользователь остается в системе. Форма содержит три поля: название, тип диаграммы и *DSL*-код. Тип выбирается из списка, по умолчанию предлагается *flowchart*. Под полями расположены кнопка «Добавить» и ссылка «Отмена».

Заполненная форма с данными новой диаграммы приведена на рисунке 3.4.

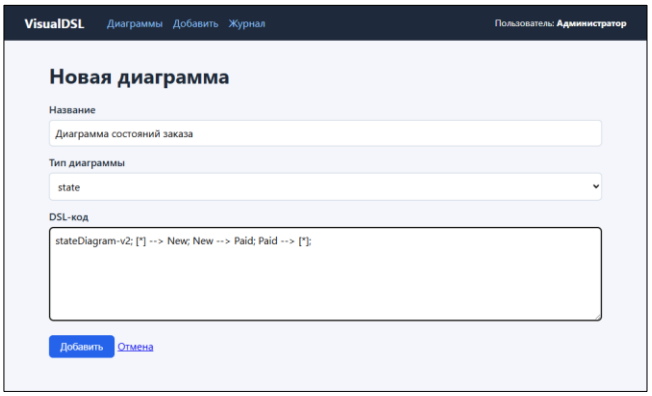


Рисунок 3.4 – Заполненная форма

Введены название «Диаграмма состояний заказа», тип *state* и *DSL*-код. После нажатия кнопки данные уходят методом *POST* на сервер, где проходят проверку. Данные еще не отправлены на сервер, поэтому в списке диаграмм новых записей пока нет. В поле *DSL*-кода записана последовательность

переходов между состояниями заказа, а тип *state* выбран в раскрывающемся списке.

Результат отправки показан на рисунке 3.5. Сервер выполнил перенаправление на главную страницу.

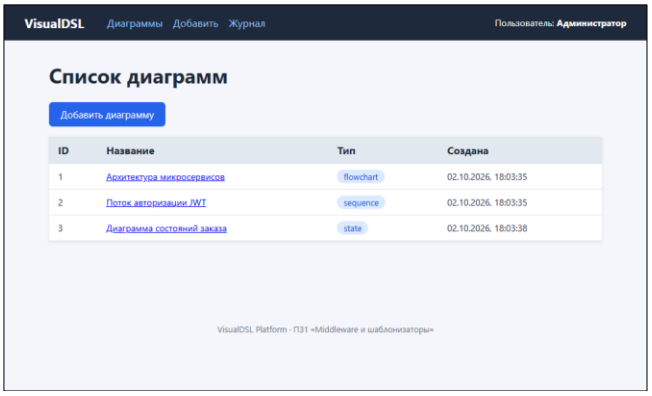


Рисунок 3.5 – Список после добавления элемента

Новая диаграмма появилась в списке третьей строкой. Это подтверждает, что *POST /add* добавил элемент в массив, а перенаправление исключает повторную отправку формы при обновлении страницы. Новая диаграмма получила следующий идентификатор и тип *state*, а дата создания проставлена автоматически при добавлении. После перенаправления адрес главной страницы сохраняет параметр авторизации, поэтому в шапке по-прежнему показан администратор.

При отправке пустой формы сервер отвечает кодом 400 и показывает список ошибок, как показано на рисунке 3.6.

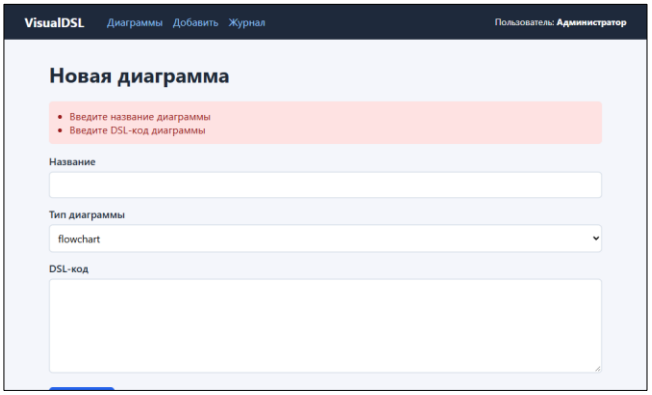


Рисунок 3.6 – Сообщения валидации формы

Проверка выполняется на стороне сервера. Пустые поля названия и *DSL*-кода отклонены, а введенные значения сохранены в форме. Массив данных при этом не изменился. Сообщения об ошибках выводятся в красной

рамке над формой: «Введите название диаграммы» и «Введите *DSL*-код диаграммы». Страница возвращается со статусом 400, а поля формы остаются доступными для исправления.

Гость, открывший */add* без параметра *auth*, попадает на страницу */login*, как показано на рисунке 3.7.

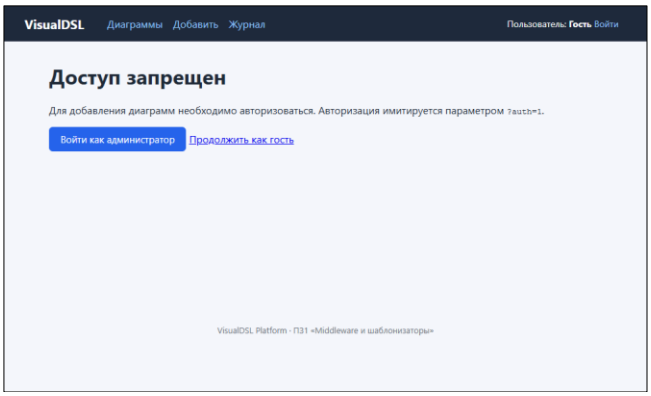


Рисунок 3.7 – Перенаправление гостя на страницу авторизации

Middleware requireAuth остановил запрос и выполнил перенаправление. Страница предлагает войти как администратор через ?*auth*=1 или продолжить как гость. Страница сообщает, что для добавления диаграмм необходимо авторизоваться, и поясняет, что авторизация имитируется параметром ?*auth*=1. Предлагаются два действия, это кнопка «Войти как администратор» и ссылка «Продолжить как гость». В шапке в этом случае показано имя пользователя «Гость» и ссылка «Войти».

Несуществующий адрес приводит к странице 404, как показано на рисунке 3.8.

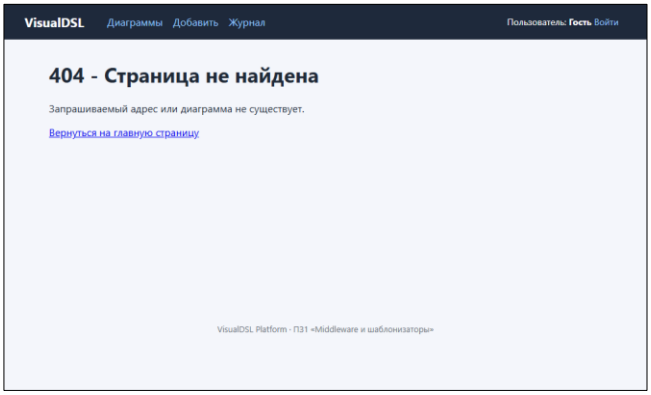


Рисунок 3.8 – Страница ошибки 404

Обработчик *notFoundHandler* сработал, потому что ни один маршрут не ответил на запрос, и отобразил отдельный шаблон 404.*ejs* с ссылкой на

главную страницу. На странице выводится заголовок «404 – Страница не найдена» и пояснение, что запрошенный адрес не существует. Запрос с таким статусом фиксируется и в журнале запросов как ошибка клиента.

Намеренно вызванное исключение на маршруте */debug/error* приводит к странице 500, как показано на рисунке 3.9.

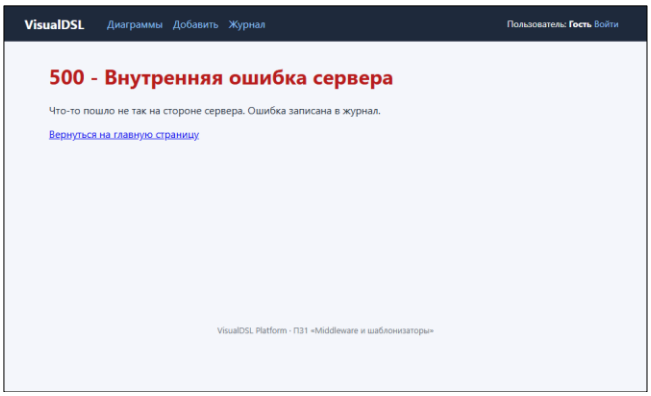


Рисунок 3.9 – Страница ошибки 500

Обработчик ошибок *serverErrorHandler* записал стек в консоль и показал пользователю безопасную страницу *500.ejs* без технических подробностей. Страница содержит заголовок «500 – Внутренняя ошибка сервера» красным цветом, пояснение, что ошибка записана в журнал, и ссылку на главную страницу. Технические подробности, такие как стек вызовов, пользователю не показываются и выводятся только в консоль сервера.

Журнал запросов, сохраненный логирующим *middleware*, показан таблицей на странице */logs*, как показано на рисунке 3.10.

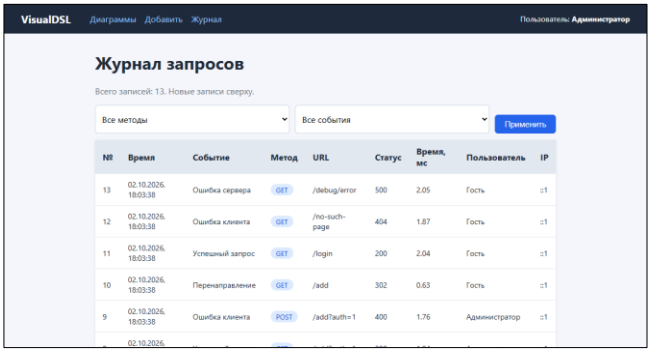


Рисунок 3.10 – Журнал запросов в виде таблицы (страница */logs*)

В таблице видны время, событие, метод, *URL*, статус, длительность, пользователь и *IP*-адрес. Запросы разных видов отличаются типом события, который может быть успешным запросом, перенаправлением, ошибкой

клиента или ошибкой сервера. Новые записи расположены сверху. Таблица журнала содержит столбцы «Время», «Событие», «Метод», «*URL*», «Статус», «Время, мс», «Пользователь» и «*IP*», а над ней расположены фильтры по методу и типу события. В верхней части страницы показано общее число записей, которое на рисунке равно 13. Строки журнала отсортированы от новых к старым. Верхняя запись с номером 13 относится к обращению к маршруту */debug/error* и имеет тип «Ошибка сервера» со статусом 500, а ниже расположены запись об ошибке клиента 404 для несуществующей страницы, успешный запрос страницы входа, перенаправление со статусом 302 при попытке открыть */add* без авторизации и ошибка 400 при отправке пустой формы.

Таблицу можно отфильтровать по методу и типу события; на рисунке 3.11 оставлены только запросы *POST*.

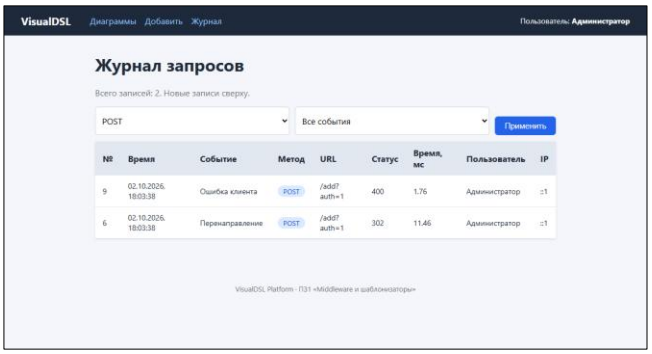


Рисунок 3.11 – Журнал с фильтром по методу *POST*

Фильтр передается параметрами строки запроса, а сервер проверяет их по списку допустимых значений, поэтому произвольный ввод игнорируется. Фильтр применен кнопкой «Применить», поэтому в таблице остались только записи с методом *POST*. Остальные столбцы и порядок записей не изменились, новые записи по-прежнему расположены сверху. Такой отбор позволяет отдельно просмотреть, например, все отправки формы и результат их проверки. Для типа события предусмотрены значения «Успешный запрос», «Перенаправление», «Ошибка клиента» и «Ошибка сервера», а допустимые значения проверяются на сервере, поэтому произвольный ввод игнорируется. Параметры фильтра передаются в адресной строке, поэтому выбранный отбор можно сохранить в закладках или отправить ссылкой.

Те же записи выводятся в консоль сервера, как показано на рисунке 3.12, что удобно при отладке.

```
Сервер запущен на http://localhost:5055
[2026-10-02T15:03:36.710Z] GET /?auth=1 -> 200 (33.92 мс)
[2026-10-02T15:03:36.765Z] GET /favicon.ico -> 404 (3.02 мс)
[2026-10-02T15:03:37.943Z] GET /item/1?auth=1 -> 200 (2.56 мс)
[2026-10-02T15:03:38.028Z] GET /add?auth=1 -> 200 (2.49 мс)
[2026-10-02T15:03:38.177Z] GET /add?auth=1 -> 304 (2.08 мс)
[2026-10-02T15:03:38.313Z] POST /add?auth=1 -> 302 (11.46 мс)
[2026-10-02T15:03:38.317Z] GET /?auth=1 -> 200 (1.83 мс)
[2026-10-02T15:03:38.390Z] GET /add?auth=1 -> 200 (1.94 мс)
[2026-10-02T15:03:38.461Z] POST /add?auth=1 -> 400 (1.76 мс)
[2026-10-02T15:03:38.519Z] GET /add -> 302 (0.63 мс)
[2026-10-02T15:03:38.523Z] GET /login -> 200 (2.04 мс)
[2026-10-02T15:03:38.586Z] GET /no-such-page -> 404 (1.87 мс)
Error: Тестовая ошибка для демонстрации страницы 500
[2026-10-02T15:03:38.643Z] GET /debug/error -> 500 (2.05 мс)
[2026-10-02T15:03:38.710Z] GET /logs?auth=1 -> 200 (2.51 мс)
[2026-10-02T15:03:38.792Z] GET /logs?auth=1&method=POST -> 200 (2.14 мс)
```

Рисунок 3.12 – Журнал запросов в консоли сервера

Каждая строка содержит время, метод, *URL*, код ответа и длительность обработки в миллисекундах.

Для проверки приложения написаны автоматические тесты на *Jest* и *Supertest*. Результаты приведены в таблице 3.1. Команда запуска тестов – *npm test*.

Таблица 3.1 – Результаты автоматического тестирования

Набор тестов	Что проверяется	Тестов
Api.test.js	*REST API /api/diagrams*: *CRUD*, поиск, валидация, *JSON*-ошибки	25
Pages.test.js	*EJS*-страницы, авторизация, форма, журнал */logs*, 404 и 500, статика, *XSS*	30
Middleware.test.js	*Logger*, *logStore*, *attachUser*, *requireAuth*, обработчики ошибок, хранилище	21
Seeds.test.js	Начальные данные, фильтры журнала, *production*, запуск *server.js*, сквозной сценарий	14
Branches.test.js	Редкие ветви: пустое тело запроса, ошибка без стека	5
Extras/__tests__	Заготовки для защиты: блокировка *IP*, *CSRF*, поиск, редактирование и другие	14
Src/__tests__	Тесты *React*-компонентов предыдущих работ	8
Итого	Все наборы пройдены успешно	117

Покрытие серверного кода, измеренное командой *npm run test*:*coverage*, составило 100 % по строкам, функциям и операторам и 96,5 % по ветвям.

4 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

25. Как реализовать защиту от *CSRF* с помощью *middleware* (теоретически)?

Атака *CSRF* (межсайтовая подделка запроса) заключается в том, что вредоносная страница заставляет браузер пользователя отправить запрос на сайт, где он уже авторизован, и браузер автоматически прикладывает *cookie* сессии. Защиту с помощью *middleware* строят на одноразовом токене. При выдаче формы сервер генерирует случайный токен, сохраняет его в сессии и вставляет скрытым полем в форму. Отдельное *middleware* при каждом запросе методом *POST*, *PUT* или *DELETE* сравнивает токен из тела запроса с сохраненным в сессии и при несовпадении отклоняет запрос с кодом 403. Дополнительно используют *cookie* с атрибутом *SameSite*, который запрещает браузеру отправлять их при запросах с чужих сайтов. Готовое решение – пакет *csurf* или его аналоги.

26. Как в *EJS* использовать функцию-*helper*?

Функция-помощник – это обычная функция, которая делается доступной во всех шаблонах, чтобы не передавать ее в каждый вызов *res.render*. В *EJS* ее кладут в *app.locals*, например *app.locals.formatDate* = (*iso*) => *new Date*(*iso*).*toLocaleString*('*ru-RU*'). Значения *app.locals* существуют все время работы приложения и доступны любому шаблону. Для данных, которые зависят от запроса, например имени пользователя, используют *res.locals*, которые заполняются в *middleware* и живут только в рамках одного запроса. В шаблоне функцию вызывают как обычную, например <%= *formatDate*(*item.createdAt*) %>, а результат выводится с экранированием. В проекте так сделано на главной и детальной страницах.

27. Почему *middleware* для статики обычно подключается до всех маршрутов?

Middleware для статических файлов подключают до маршрутов, потому что запросы к стилям, скриптам и картинкам должны обслуживаться сразу, не проходя через авторизацию, парсеры и логику маршрутов. Если подключить его после маршрутов, каждый запрос к файлу сначала будет обработан цепочкой обработчиков и лишь потом дойдет до *express.static*, что замедляет ответ и расходует ресурсы сервера. Кроме того, при неверном порядке запрос к существующему файлу может быть перехвачен обработчиком 404, и вместо файла пользователь получит страницу ошибки.

ЗАКЛЮЧЕНИЕ

В ходе работы создано *Express*-приложение с шаблонизатором *EJS*, тематика которого соответствует проекту *VisualDSL*. Реализованы *middleware* логирования с выводом журнала в таблицу (время, событие, метод, *URL*, статус и другие поля) и имитации авторизации, страницы списка, деталей и добавления элемента и отдельные страницы ошибок 404 и 500. Данные хранятся в массиве в памяти и используются как *HTML*-страницами, так и *REST API*.

Правильность работы подтверждена 117 автоматическими тестами и скриншотами страниц, а покрытие серверного кода по строкам составило 100 %. Практическим результатом стало понимание того, что порядок подключения *middleware* определяет поведение приложения. Статика и разбор тела запроса подключаются до маршрутов, а обработчики 404 и 500 – после них.

Дополнительно реализованы проверка полей формы на стороне сервера, журнал запросов с фильтрацией по методу и типу события и начальные данные для тестов и демонстрации. Исходный код приложения вынесен в приложения А–И, а результаты работы показаны на рисунках 3.1–3.12.

Все пять задач практического занятия выполнены. Создано приложение с шаблонизатором *EJS* и частичными шаблонами, реализованы *middleware* логирования и имитации авторизации, созданы динамические страницы списка, деталей и добавления, добавлены отдельные страницы ошибок 404 и 500. Работа показала, что *middleware* удобны для выделения сквозной логики, а порядок их подключения напрямую влияет на результат.

Проверка выполнена в браузере и автоматическими тестами, покрытие серверного кода составило 100 % по строкам, функциям и операторам и 96,5 % по ветвям. Журнал запросов хранит не более 500 записей и выводится таблицей с фильтрацией по методу и типу события. Страницы ошибок 404 и 500 показываются пользователю без технических подробностей, а для запросов к *API* возвращается *JSON*.

Структура приложения, в которой точка входа *server.js* отделена от сборки *app.js*, а код разделен на *middlewares*, *controllers*, *models*, *routes* и *views*, позволяет подключать приложение в тестах без запуска сервера и расширять его новыми страницами и обработчиками. Полученные навыки применимы при разработке серверной части курсового проекта, тематика которого совпадает с тематикой приложения.

ПРИЛОЖЕНИЕ А
(обязательное)
Настройка *Express* (*app.js*)

```
const path = require('path');
const express = require('express');
const logger = require('./middlewares/logger');
const { attachUser } = require('./middlewares/auth');
const { notFoundHandler, serverErrorHandler } =
require('./middlewares/errorHandlers');
const pageRoutes = require('./routes/pageRoutes');
const diagramRoutes = require('./routes/diagramRoutes');

const app = express();

// Шаблонизатор EJS.
app.set('view engine', 'ejs');
app.set('views', path.join(__dirname, 'views'));

// Helper для шаблонов: форматирование даты.
app.locals.formatDate = (iso) => new Date(iso).toLocaleString('ru-RU');

// Статика подключается до маршрутов, чтобы файлы отдавались без лишних
проверок.
app.use('/static', express.static(path.join(__dirname, 'public')));

app.use(logger);

app.use(express.urlencoded({ extended: true }));
app.use(express.json());
app.use(attachUser);

app.use('/', pageRoutes);
app.use('/api/diagrams', diagramRoutes);

// Обработчики ошибок идут последними.
app.use(notFoundHandler);
app.use(serverErrorHandler);

module.exports = app;
```

ПРИЛОЖЕНИЕ Б
(обязательное)
Шаблон главной страницы (*index.ejs*)

```
<%- include('partials/header')  %>
<h1>Список диаграмм</h1>
<p><a class="btn" href="/add<%= authQuery %>">Добавить диаграмму</a></p>

<% if (items.length  === 0) { %>
    <p class="muted">Диаграмм пока нет.</p>
<% } else { %>
    <table>
        <thead>
            <tr><th>ID</th><th>Название</th><th>Тип</th><th>Создана</th>
            </tr>
        </thead>
        <tbody>
            <% items.forEach(function  (item) { %>
                <tr>
                    <td><%= item.id %></td>
                    <td><a href="/item/<%=  item.id %><%= authQuery
                    %>"><%= item.title %></a></td>
                    <td><span class="badge"><%= item.type %></span></td>
                    <td><%= formatDate(item.createdAt)  %></td>
                </tr>
            <% }); %>
        </tbody>
    </table>
<% } %>
<%- include('partials/footer')  %>
```

ПРИЛОЖЕНИЕ В
(обязательное)
Шаблон шапки (*header.ejs*)

```
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,  initial-scale=1.0">
    <title><%= title %> | VisualDSL</title>
    <link rel="stylesheet"  href="/static/css/style.css">
</head>
<body>
<header class="site-header">
    <a class="brand" href="/<%= authQuery %>">VisualDSL</a>
    <nav>
        <a href="/<%= authQuery %>">Диаграммы</a>
        <a href="/add<%= authQuery %>">Добавить</a>
        <a href="/logs<%= authQuery %>">Журнал</a>
    </nav>
    <span class="user">Пользователь: <strong><%= user.name %></strong>
        <% if (!user.isAuth) { %><a href="/login">Войти</a><% } %>
    </span>
</header>
<main class="container">
```

ПРИЛОЖЕНИЕ Г
(обязательное)
Логирование запросов (*logger.js*)

```
// Логирование: метод, URL, статус и время обработки (мс) после отправки ответа.
// Запись попадает в журнал (таблица на странице /logs) и в консоль (кроме тестов).
const logStore = require('../models/logStore');

module.exports = (req, res, next) => {
  const start = process.hrtime.bigint();

  res.on('finish', () => {
    const ms = Number(process.hrtime.bigint() - start) / 1e6;
    const entry = logStore.add({
      method: req.method,
      url: req.originalUrl,
      status: res.statusCode,
      durationMs: ms,
      user: req.user ? req.user.name : 'Гость',
      ip: req.ip || '-'
    });

    if (process.env.NODE_ENV !== 'test') {
      console.log(
        `[${entry.time}] ${entry.method} ${entry.url} -> ${entry.status}
        (${entry.durationMs.toFixed(2)} мс)`
      );
    }
  });

  next();
};
```

ПРИЛОЖЕНИЕ Д
(обязательное)
Журнал запросов (*logStore.js*)

```
// Журнал запросов в памяти: записи отображаются на странице /logs в виде таблицы.
const MAX_ENTRIES = 500;

let entries = [];
let nextId = 1;

// Тип события определяется по коду ответа.
const eventByStatus = (status) => {
  if (status >= 500) return 'Ошибка сервера';
  if (status >= 400) return 'Ошибка клиента';
  if (status >= 300) return 'Перенаправление';
  return 'Успешный запрос';
};

const add = ({ method, url, status, durationMs, user, ip }) => {
  const entry = {
    id: nextId++,
    time: new Date().toISOString(),
    event: eventByStatus(status),
    method,
    url,
    status,
    durationMs: Number(durationMs.toFixed(2)),
    user,
    ip
  };
  entries.push(entry);
  if (entries.length > MAX_ENTRIES) entries.shift();
  return entry;
};

// Новые записи первыми; необязательные фильтры по методу и классу события.
const getAll = ({ method, event } = {}) =>
  entries
    .filter((e) => (!method || e.method === method) && (!event || e.event === event))
    .reverse();

const clear = () => {
  entries = [];
  nextId = 1;
};

module.exports = { MAX_ENTRIES, eventByStatus, add, getAll, clear };
```

ПРИЛОЖЕНИЕ Е
(обязательное)
Имитация авторизации (*auth.js*)

```
// Имитация авторизации по query-параметру ?auth=1 (без сессий и паролей).
const GUEST = { name: 'Гость', isAuth: false };
const ADMIN = { name: 'Администратор', isAuth: true };

// Определяет пользователя и передает его в шаблоны (res.locals).
const attachUser = (req, res, next) => {
  req.user = req.query.auth === '1' ? ADMIN : GUEST;
  res.locals.user = req.user;
  // Суффикс для ссылок, чтобы "авторизация" сохранялась при переходах по
  сайту.
  res.locals.authQuery = req.user.isAuth ? '?auth=1' : '';
  next();
};

// Защита маршрута: гость перенаправляется на /login.
const requireAuth = (req, res, next) => {
  if (req.user && req.user.isAuth) return next();
  res.redirect('/login');
};

module.exports = { attachUser, requireAuth };
```

ПРИЛОЖЕНИЕ Ж
(обязательное)
Маршруты страниц (*pageRoutes.js*)

```
const express = require('express');
const { requireAuth } = require('../middlewares/auth');
const pages = require('../controllers/pageController');

const router = express.Router();

router.get('/', pages.index);
router.get('/item/:id', pages.item);
router.get('/login', pages.login);
router.get('/logs', pages.logs);
// Middleware requireAuth выполняется только для этих маршрутов.
router.get('/add', requireAuth, pages.addForm);
router.post('/add', requireAuth, pages.addSubmit);

if (process.env.NODE_ENV !== 'production') {
  router.get('/debug/error', pages.crash);
}

module.exports = router;
```

ПРИЛОЖЕНИЕ И
(обязательное)
Обработка ошибок (*errorHandlers.js*)

```
// Обработчики ошибок: 404 (маршрут не найден) и 500 (исключение в
приложении).
const wantsJson = (req) => req.originalUrl.startsWith('/api/');

const notFoundHandler = (req, res) => {
  if (wantsJson(req)) return res.status(404).json({ error: 'Маршрут не найден' });
  res.status(404).render('404', { title: 'Ошибка 404' });
};

// Четыре параметра - признак error-handling middleware в Express.
const serverErrorHandler = (err, req, res, next) => {
  console.error(err.stack || err);
  if (res.headersSent) return next(err);
  if (wantsJson(req)) {
    return res.status(500).json({ error: 'Внутренняя ошибка сервера', message:
    err.message });
  }
  res.status(500).render('500', { title: 'Ошибка 500' });
};

module.exports = { notFoundHandler, serverErrorHandler };
```